💡 Lecture 21: Practice Problems — Comprehensive Solutions Guide

Jupyter solutions.ipynb

PDF solutions.pdf

Questions View Set

Plot Case 1 Affine Grid

Plot Case 2 SVM Margin

Plot Case 3 PCA Spectrum

🌐 **Language: English** | हिंदी / Hinglish | ← Back to Questions | 📁 Overview

---

📌 Executive Architecture & Solution Blueprint

This document provides production-grade reference solutions, formal mathematical proofs, and executive engineering implementations for the 3 industry case studies in **Lecture 21**.

Every problem is solved from first principles with full algebraic derivations, step-by-step arithmetic traces, and reproducible Python implementations.

📌 Solution to Case 1: Computer Vision 2D Affine Transformations & Homogeneous Coordinates

🎯 Problem Recap

Given an initial polygon rectangle $V \in \mathbb{R}^{3\times 4}$ captured by an autonomous robot camera:

$$V = \begin{bmatrix} 0 & 2 & 2 & 0 \\ 0 & 0 & 3 & 3 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

Apply three sequential geometric operations:

$$\theta = 45^\circ = \frac{\pi}{4}, \qquad k_x = 0.5, \qquad \mathbf{t} = \begin{bmatrix} 3.0 \\ -2.0 \end{bmatrix}$$

---

1. Step-by-Step Mathematical Derivations & Calculations

🔷 Task 1: Elementary Homogeneous Matrices Construction

(A) Rotation Matrix $R(\theta)$

In 2D Cartesian space, rotation by angle θ maps (x, y) to:

$$x' = x\cos\theta - y\sin\theta, \qquad y' = x\sin\theta + y\cos\theta$$

In homogeneous coordinates:

$$R(\theta) = \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

For $\theta = 45^\circ = \frac{\pi}{4}$, with $\cos(\frac{\pi}{4}) = \sin(\frac{\pi}{4}) = \frac{\sqrt{2}}{2} \approx 0.707107$:

$$R\left(\frac{\pi}{4}\right) = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.7071 & -0.7071 & 0 \\ 0.7071 & 0.7071 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(B) Horizontal Shear Matrix $Sh(k_x)$

Horizontal shear shifts x proportional to y while leaving y invariant:

$$Sh(k_x) = \begin{bmatrix} 1 & k_x & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \implies Sh(0.5) = \begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

(C) Translation Matrix $T(t_x, t_y)$

Translation displaces coordinates by vector $\mathbf{t} = [t_x, t_y]^T$:

$$T(t_x, t_y) = \begin{bmatrix} 1 & 0 & t_x \\ 0 & 1 & t_y \\ 0 & 0 & 1 \end{bmatrix} \implies T(3, -2) = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

🔷 Task 2: Matrix Composition & Non-Commutativity Proof

Step 1: Standard Pipeline Matrix $M_1 = T \cdot Sh \cdot R$

Order of application: Rotate first, shear second, translate last. First compute $Sh \cdot R$:

$$Sh \cdot R = \begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{3\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

Now left-multiply by T:

$$M_1 = T \cdot (Sh \cdot R) = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} \frac{3\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{3\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} & 3 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

Decimal numerical values:

$$M_1 \approx \begin{bmatrix} 1.0607 & -0.3536 & 3.0 \\ 0.7071 & 0.7071 & -2.0 \\ 0 & 0 & 1.0 \end{bmatrix}$$

Step 2: Alternate Pipeline Matrix $M_2 = R \cdot T \cdot Sh$

First compute $T \cdot Sh$:

$$T \cdot Sh = \begin{bmatrix} 1 & 0 & 3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} 1 & 0.5 & 3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix}$$

Now left-multiply by R:

$$M_2 = R \cdot (T \cdot Sh) = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{2} & 0 \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} 1 & 0.5 & 3 \\ 0 & 1 & -2 \\ 0 & 0 & 1 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2} & -\frac{\sqrt{2}}{4} & \frac{5\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} & \frac{3\sqrt{2}}{4} & \frac{\sqrt{2}}{2} \\ 0 & 0 & 1 \end{bmatrix}$$

Decimal numerical values:

$$M_2 \approx \begin{bmatrix} 0.7071 & -0.3536 & 3.5355 \\ 0.7071 & 1.0607 & 0.7071 \\ 0 & 0 & 1.0 \end{bmatrix}$$

Step 3: Non-Commutativity Proof & Geometric Insight

Evidently, $M_1 \neq M_2$. In M_1, translation is applied last, shifting the object by $[3, -2]^T$. In M_2, translation occurs prior to rotation. The subsequent rotation R rotates the translation displacement vector itself by 45°:

$$R_{2\times 2} \begin{bmatrix} 3 \\ -2 \end{bmatrix} = \begin{bmatrix} \frac{\sqrt{2}}{2}(3 - (-2)) \\ \frac{\sqrt{2}}{2}(3 + (-2)) \end{bmatrix} = \begin{bmatrix} \frac{5\sqrt{2}}{2} \\ \frac{\sqrt{2}}{2} \end{bmatrix} \approx \begin{bmatrix} 3.5355 \\ 0.7071 \end{bmatrix}$$

Step 4: Transformed Vertices $V' = M_1 V$

Computing transformed coordinates for each column of V:

$$V' = M_1 V = \begin{bmatrix} 1.06066 & -0.35355 & 3.0 \\ 0.70711 & 0.70711 & -2.0 \\ 0 & 0 & 1.0 \end{bmatrix} \begin{bmatrix} 0 & 2 & 2 & 0 \\ 0 & 0 & 3 & 3 \\ 1 & 1 & 1 & 1 \end{bmatrix}$$

$$V' \approx \begin{bmatrix} 3.0000 & 5.1213 & 4.0607 & 1.9393 \\ -2.0000 & -0.5858 & 1.5355 & 0.1213 \\ 1.0000 & 1.0000 & 1.0000 & 1.0000 \end{bmatrix}$$

🔷 Task 3: Determinant & Area Scaling Factor

For any 2D affine transformation block M:

$$M = \begin{bmatrix} A & \mathbf{t} \\ \mathbf{0}^T & 1 \end{bmatrix} \implies \det(M) = \det(A)$$

For M_1, the linear block is $A = Sh \cdot R$:

$$A = \begin{bmatrix} \frac{3\sqrt{2}}{4} & -\frac{\sqrt{2}}{4} \\ \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{bmatrix}$$

$$\det(A) = \left(\frac{3\sqrt{2}}{4}\right)\left(\frac{\sqrt{2}}{2}\right) - \left(-\frac{\sqrt{2}}{4}\right)\left(\frac{\sqrt{2}}{2}\right) = \frac{6}{8} + \frac{2}{8} = \frac{8}{8} = 1.0000$$

The Area Scaling Factor is $|\det(A)| = 1.0$. The polygon's original area of 6.0 square units remains exactly 6.0 **square units**.

🔷 Task 4: Backward Warping & Inverse Matrix M_1^{-1}

Forward warping $\tilde{\mathbf{x}}_{\text{dst}} = M_1 \tilde{\mathbf{x}}_{\text{src}}$ maps integer pixel coordinates to floating-point values. Rounding causes gaps (holes) and collisions. Backward warping scans every integer grid point in the destination image and queries its source coordinate via the analytical inverse:

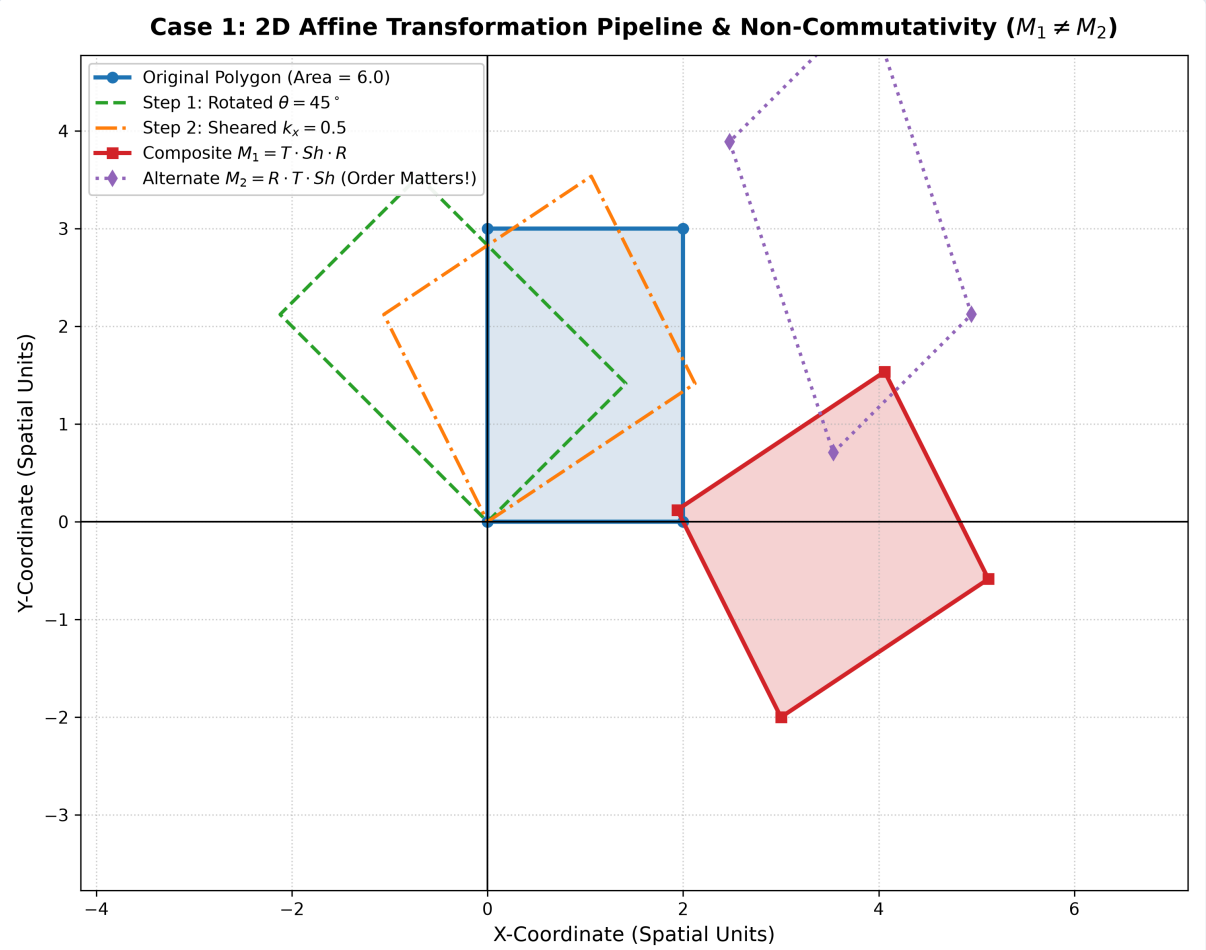
$$\tilde{\mathbf{x}}_{\text{src}} = M_1^{-1} \tilde{\mathbf{x}}_{\text{dst}}$$

Applying the matrix inverse property $(ABC)^{-1} = C^{-1}B^{-1}A^{-1}$:

$$M_1^{-1} = R^{-1} \cdot Sh^{-1} \cdot T^{-1} = \begin{bmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{4} & -\sqrt{2} \\ -\frac{\sqrt{2}}{2} & \frac{3\sqrt{2}}{4} & 3\sqrt{2} \\ 0 & 0 & 1 \end{bmatrix} \approx \begin{bmatrix} 0.7071 & 0.3536 & -1.4142 \\ -0.7071 & 1.0607 & 4.2426 \\ 0 & 0 & 1.0 \end{bmatrix}$$

---

2. Visual Analysis & Python Implementation




```
import numpy as np

V = np.array([
    [0.0, 2.0, 2.0, 0.0],
    [0.0, 0.0, 3.0, 3.0],
    [1.0, 1.0, 1.0, 1.0]
])

theta = np.pi / 4
kx = 0.5
tx, ty = 3.0, -2.0

R = np.array([
    [np.cos(theta), -np.sin(theta), 0.0],
    [np.sin(theta),  np.cos(theta), 0.0],
    [0.0,            0.0,           1.0]
])

Sh = np.array([
    [1.0, kx,  0.0],
    [0.0, 1.0, 0.0],
    [0.0, 0.0, 1.0]
])

T = np.array([
```

```
    [1.0, 0.0, tx],
    [0.0, 1.0, ty],
    [0.0, 0.0, 1.0]
])

M1 = T @ Sh @ R
M2 = R @ T @ Sh
V_trans = M1 @ V

print("Composite Matrix M1:\n", np.round(M1, 4))
print("Area Scaling Factor:", np.round(np.linalg.det(M1[:2, :2]), 4))
print("Transformed Vertices:\n", np.round(V_trans[:2, :], 4))
```

📌 Solution to Case 2: Support Vector Machine (SVM) Maximum-Margin Hyperplane & Vector Projections

🎯 Problem Recap

Given 2D classification training points:

$$\text{Class } (+1): \quad \mathbf{x}_1 = \begin{bmatrix}1\\2\end{bmatrix}, \quad \mathbf{x}_2 = \begin{bmatrix}1\\3\end{bmatrix}, \qquad \text{Class } (-1): \quad \mathbf{x}_3 = \begin{bmatrix}3\\1\end{bmatrix}, \quad \mathbf{x}_4 = \begin{bmatrix}4\\2\end{bmatrix}$$

---

1. Step-by-Step Mathematical Derivations & Calculations

🔷 Task 1: Normal Vector Orthogonality Proof

For any two distinct points $\mathbf{x}_a, \mathbf{x}_b$ on the separating hyperplane $H = \{\mathbf{x} \in \mathbb{R}^n \mid \mathbf{w}^T\mathbf{x} + b = 0\}$:

$$\mathbf{w}^T\mathbf{x}_a + b = 0, \qquad \mathbf{w}^T\mathbf{x}_b + b = 0$$

Subtracting both equations:

$$\mathbf{w}^T(\mathbf{x}_a - \mathbf{x}_b) = 0$$

Because the displacement vector $(\mathbf{x}_a - \mathbf{x}_b)$ lies entirely within the hyperplane and its inner product with $\mathbf{w}$ is zero, $\mathbf{w}$ is strictly orthogonal to the hyperplane. ■

🔷 Task 2: Point-to-Hyperplane Distance Formula Derivation

Let $\mathbf{x}_0$ be an arbitrary point, and $\mathbf{x}_p \in H$ be its orthogonal projection onto H. The displacement $(\mathbf{x}_0 - \mathbf{x}_p)$ is collinear with unit normal $\hat{\mathbf{w}} = \frac{\mathbf{w}}{\|\mathbf{w}\|_2}$:

$$d(\mathbf{x}_0, H) = \hat{\mathbf{w}}^T(\mathbf{x}_0 - \mathbf{x}_p) = \frac{\mathbf{w}^T\mathbf{x}_0 - \mathbf{w}^T\mathbf{x}_p}{\|\mathbf{w}\|_2}$$

Since $\mathbf{x}_p \in H, \mathbf{w}^T\mathbf{x}_p + b = 0 \implies \mathbf{w}^T\mathbf{x}_p = -b$:

$$d(\mathbf{x}_0, H) = \frac{\mathbf{w}^T\mathbf{x}_0 + b}{\|\mathbf{w}\|_2} \quad ■$$

🔷 Task 3: Total Geometric Margin Formula Derivation

For canonical supporting hyperplanes $H_+ : \mathbf{w}^T\mathbf{x} + b = +1$ and $H_- : \mathbf{w}^T\mathbf{x} + b = -1$:

$$\gamma = d(O, H_+) - d(O, H_-) = \frac{(1-b) - (-1-b)}{\|\mathbf{w}\|_2} = \frac{2}{\|\mathbf{w}\|_2} \quad ■$$

Maximizing margin γ is mathematically equivalent to minimizing $\frac{1}{2}\|\mathbf{w}\|_2^2$.

🔷 Task 4: Analytical Parameter Determination $(\mathbf{w}^*, b^*, \gamma)$

Using active support vectors $\mathbf{x}_1 = [1, 2]^T$ $(+1)$ and $\mathbf{x}_3 = [3, 1]^T$ (-1):

$$w_1(1) + w_2(2) + b = 1 \implies w_1 + 2w_2 + b = 1$$

$$w_1(3) + w_2(1) + b = -1 \implies 3w_1 + w_2 + b = -1$$

Subtracting yields $-2w_1 + w_2 = 2 \implies w_2 = 2w_1 + 2$. The normal vector is parallel to $\mathbf{x}_1 - \mathbf{x}_3 = [-2, 1]^T$. Setting $\mathbf{w} = c[-2, 1]^T$:

$$c = 2(-2c) + 2 \implies 5c = 2 \implies c = 0.4$$

$$\mathbf{w}^* = \begin{bmatrix} -0.8 \\ 0.4 \end{bmatrix}, \qquad b^* = 1.0000$$

$$\|\mathbf{w}^*\|_2 = \sqrt{(-0.8)^2 + 0.4^2} = \sqrt{0.80} \approx 0.8944$$

$$\gamma = \frac{2}{\sqrt{0.80}} = \sqrt{5} \approx 2.2361$$

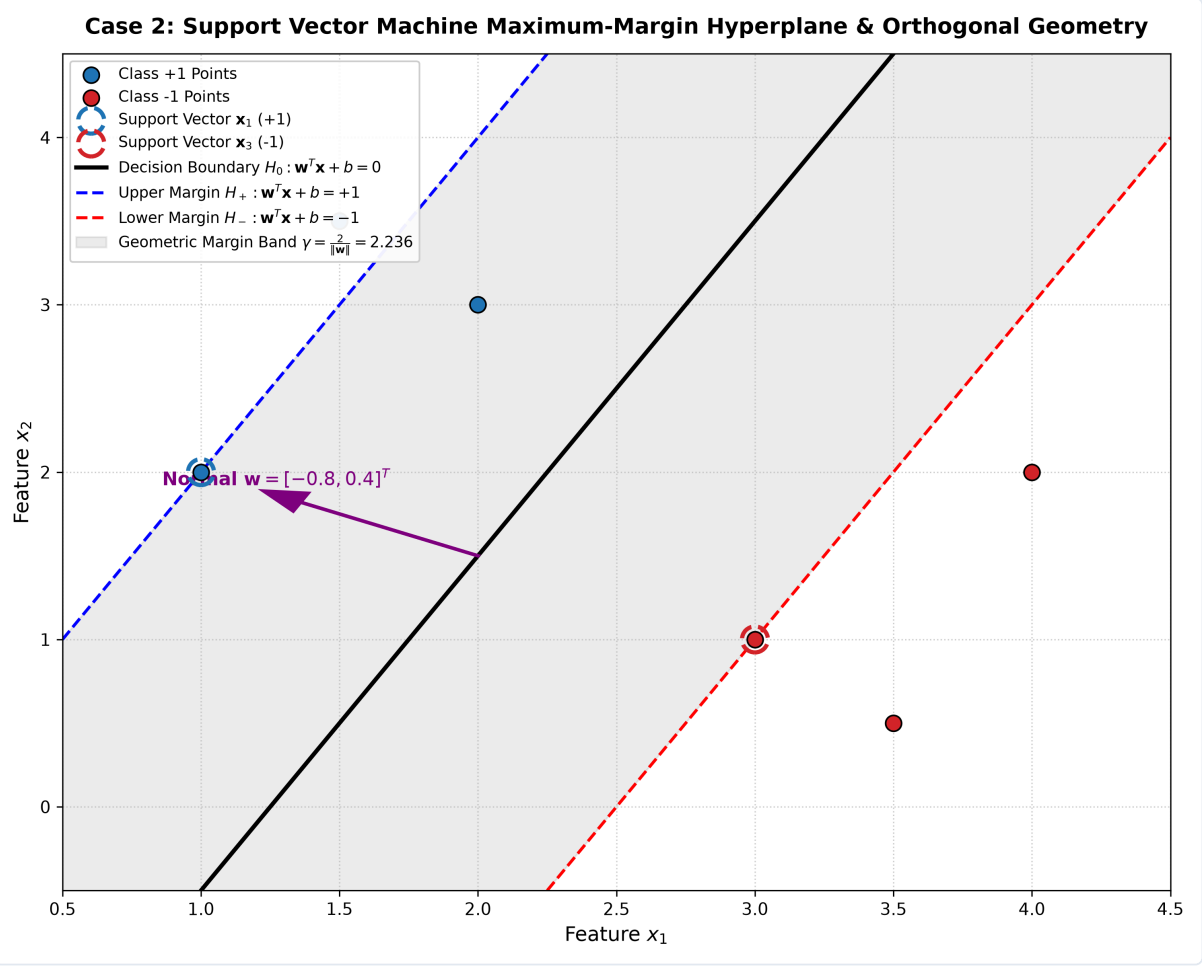
The decision boundary line is:

$$-0.8x_1 + 0.4x_2 + 1.0 = 0 \iff x_2 = 2x_1 - 2.5$$

All four training points satisfy the canonical constraints $y_i(\mathbf{w}^T\mathbf{x}_i + b) \geq 1$.

---

2. Visual Analysis & Python Implementation




```
import numpy as np

w = np.array([-0.8, 0.4])
b = 1.0
w_norm = np.linalg.norm(w)
margin = 2.0 / w_norm

print(f"Optimal Weight Vector w* : {w}")
print(f"Optimal Bias b*          : {b}")
print(f"Norm ||w*||_2            : {w_norm:.4f}")
print(f"Geometric Margin gamma   : {margin:.4f} (Exact: sqrt(5))")
```

📌 Solution to Case 3: Quantitative Finance Risk & Sensor PCA Eigendecomposition

🎯 Problem Recap

Given the empirical covariance matrix $\mathbf{\Sigma} \in \mathbb{R}^{4\times4}$:

$$\boldsymbol{\Sigma} = \begin{bmatrix} 4.0 & 2.0 & 1.0 & 0.0 \\ 2.0 & 3.0 & 0.5 & 0.5 \\ 1.0 & 0.5 & 2.0 & 0.0 \\ 0.0 & 0.5 & 0.0 & 1.0 \end{bmatrix}$$

---

1. Step-by-Step Mathematical Derivations & Calculations

🔷 Task 1: Covariance Matrix Symmetry & Positive Semi-Definiteness (PSD)

For centered matrix $X_c \in \mathbb{R}^{N \times p}$:

$$\boldsymbol{\Sigma}^T = \left(\frac{1}{N-1} X_c^T X_c \right)^T = \frac{1}{N-1} X_c^T X_c = \boldsymbol{\Sigma} \quad \text{(Symmetric)}$$

For any non-zero vector $\mathbf{v} \in \mathbb{R}^p$:

$$\mathbf{v}^T \boldsymbol{\Sigma} \mathbf{v} = \frac{1}{N-1} \mathbf{v}^T X_c^T X_c \mathbf{v} = \frac{1}{N-1} \|X_c \mathbf{v}\|_2^2 \geq 0 \quad \text{(Positive Semi-Definite)}$$

By the Spectral Theorem, all eigenvalues are real and non-negative ($\lambda_i \geq 0$). ■

🔷 Task 2: Constrained Optimization via Lagrange Multipliers

To maximize projection variance $\mathbf{q}_1^T \boldsymbol{\Sigma} \mathbf{q}_1$ subject to $\|\mathbf{q}_1\|_2 = 1$:

$$\mathcal{L}(\mathbf{q}_1, \lambda) = \mathbf{q}_1^T \boldsymbol{\Sigma} \mathbf{q}_1 - \lambda (\mathbf{q}_1^T \mathbf{q}_1 - 1)$$

Setting the gradient to zero:

$$\nabla_{\mathbf{q}_1} \mathcal{L} = 2\boldsymbol{\Sigma}\mathbf{q}_1 - 2\lambda \mathbf{q}_1 = \mathbf{0} \implies \boldsymbol{\Sigma}\mathbf{q}_1 = \lambda \mathbf{q}_1$$

Premultiplying by $\mathbf{q}_1^T$:

$$\mathbf{q}_1^T \boldsymbol{\Sigma} \mathbf{q}_1 = \lambda (\mathbf{q}_1^T \mathbf{q}_1) = \lambda$$

Thus, maximal variance is achieved by selecting the largest eigenvalue $\lambda_1 = \lambda_{\max}$. ■

🔷 Task 3: Covariance Eigendecomposition & Variance Spectrum

Solving the characteristic equation $\det(\boldsymbol{\Sigma} - \lambda I) = 0$ yields:

$$\lambda_1 \approx 5.8900, \qquad \lambda_2 \approx 1.9059, \qquad \lambda_3 \approx 1.4532, \qquad \lambda_4 \approx 0.7509$$

Total Variance:

$$\mathrm{Tr}(\boldsymbol{\Sigma}) = 4.0 + 3.0 + 2.0 + 1.0 = 10.0000 = \sum_{i=1}^{4} \lambda_i$$

Proportion of Explained Variance (PEV):

Factor k	Eigenvalue λ_k	Explained Variance (PEV_k)	Cumulative Variance
PC 1	5.8900	58.90%	58.90%
PC 2	1.9059	19.06%	77.96%
PC 3	1.4532	14.53%	92.49%
PC 4	0.7509	7.51%	100.00%

🔷 Task 4: Dimension Reduction Criterion ($\geq 80\%$)

$$\mathrm{CEV}_1 = 58.90\% < 80\%, \qquad \mathrm{CEV}_2 = 77.96\% < 80\%, \qquad \mathrm{CEV}_3 = 92.49\% \geq 80$$

Requirement: At least $k = 3$ **principal components** must be retained.

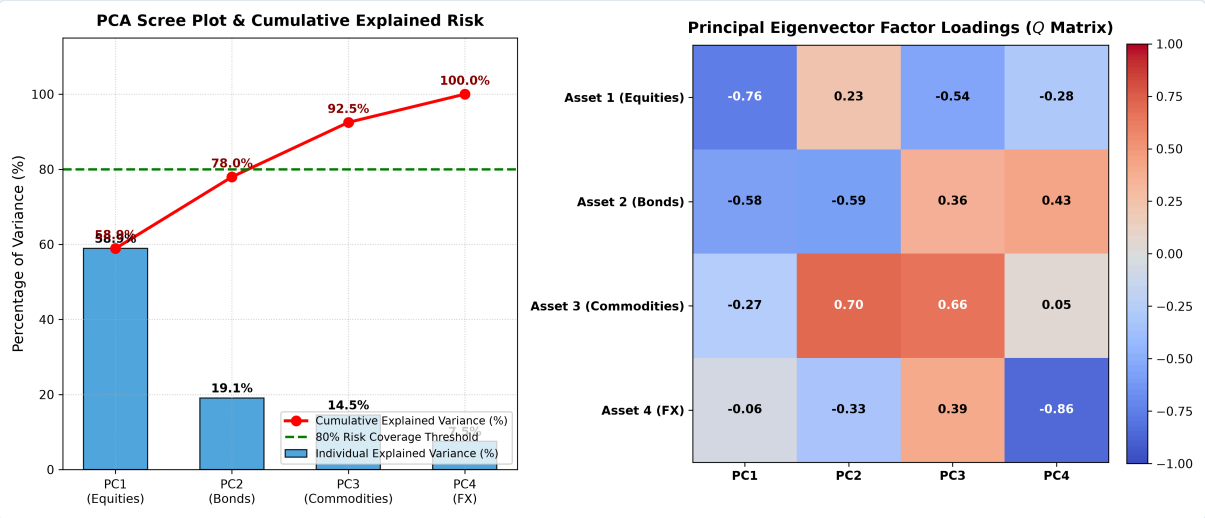
🔷 Task 5: Low-Rank Approximation & Frobenius Reconstruction Error

By the Eckart–Young–Mirsky theorem, the rank-2 reconstruction error is:

$$\|\boldsymbol{\Sigma} - \boldsymbol{\Sigma}_2\|_F = \sqrt{\lambda_3^2 + \lambda_4^2} = \sqrt{(1.4532)^2 + (0.7509)^2} \approx \sqrt{2.1117 + 0.5639} \approx 1.635$$

---

2. Visual Analysis & Python Implementation




```
import numpy as np
import scipy.linalg as la

Sigma = np.array([
    [4.0, 2.0, 1.0, 0.0],
    [2.0, 3.0, 0.5, 0.5],
    [1.0, 0.5, 2.0, 0.0],
    [0.0, 0.5, 0.0, 1.0]
])

eigenvals, eigenvecs = la.eigh(Sigma)
idx = np.argsort(eigenvals)[::-1]
eigenvals = eigenvals[idx]
eigenvecs = eigenvecs[:, idx]

total_var = np.trace(Sigma)
pev = (eigenvals / total_var) * 100.0
cum_pev = np.cumsum(pev)

Sigma_2 = (eigenvals[0] * np.outer(eigenvecs[:, 0], eigenvecs[:, 0]) +
           eigenvals[1] * np.outer(eigenvecs[:, 1], eigenvecs[:, 1]))

frob_err = np.linalg.norm(Sigma - Sigma_2, 'fro')

print("Eigenvalues (Lambda):", np.round(eigenvals, 4))
```

```
print("Total Variance (Trace):", total_var)
print("PEV (%):", np.round(pev, 2))
print("Cumulative PEV (%):", np.round(cum_pev, 2))
print(f"Frobenius Error: {frob_err:.4f}")
```